Applied Mathematical Modelling 151 (2026) 116477

Contents lists available at ScienceDirect

Applied Mathematical Modelling

journal homepage: www.elsevier.com/locate/apm

Deep Gaussian Process model and its application in active learning reliability analysis

Yuan Lu [a,b], Yuxin Liu [a,b], Hong-Zhong Huang [a,b,*], Yan-Feng Li [a,b], Zhe Deng [a,b]

[a] School of Mechanical and Electrical Engineering, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, PR China
[b] Center for System Reliability and Safety, University of Electronic Science and Technology of China, Chengdu, Sichuan, 611731, PR China

ARTICLE INFO

Keywords:
Deep Gaussian Processes
MATLAB toolbox
Engineering machinery
Reliability analysis

ABSTRACT

Reliability analysis of engineering machinery faces significant challenges due to highdimensional input spaces and scarce failure samples, making accurate reliability assessment difficult. Recognizing the potential of Deep Gaussian Processes in this domain, this paper proposes a surrogate modeling method integrating Deep Gaussian Processes with active learning strategies and develops a dedicated toolbox. The toolbox employs a modular architecture to enable scalable layer-wise operations and includes computational efficiency benchmarks. Through two structural reliability case studies, including a numerical example with explicit limit states and a practical engineering application requiring finite element analysis, the proposed method demonstrates superiority over traditional surrogate models in terms of sample efficiency and prediction accuracy. The findings highlight the practical utility of Deep Gaussian Processes in advancing reliability engineering practices.

1. Introduction

Surrogate modeling remains a mainstream approach for structural reliability assessment, with widely adopted techniques including response surface methods [1], polynomial chaos expansion [2], and Kriging [3]. Among these, active learning Kriging has become the dominant strategy due to its adaptability and efficiency [4]. To address increasingly complex limit-state functions and computationally expensive finite element analysis, researchers have developed various refinements in active learning strategies [5], convergence criteria [6], and sampling methods [7], effectively reducing the number of high-fidelity simulations while maintaining accuracy. However, Kriging models, grounded in conventional Gaussian processes (GP), still struggle to capture non-stationary and discontinuous responses common in modern engineering systems, such as those involving localized stress concentrations or abrupt failure boundaries. Although efforts like localized GP approximations [8], partitioned GP models [9], and sparse covariance structures [10] have been proposed, they often introduce trade-offs between computational efficiency and predictive fidelity. These persistent challenges shift the focus from external learning strategies to the surrogate model itself, underscoring the need for a more flexible and expressive modeling framework.

Deep Gaussian Processes (DGP), introduced by Damianou and Lawrence [11], offers a principled approach to modeling nonstationary systems through hierarchical compositions of Gaussian mappings. This architecture natively captures heterogeneous smoothness and discontinuous transitions by implicitly warping the input space via latent layers, eliminating the need for ad hoc

* Corresponding author.
E-mail address: hzhuang@uestc.edu.cn (H.-Z. Huang).

https://doi.org/10.1016/j.apm.2025.116477
Received 6 August 2025; Received in revised form 21 September 2025; Accepted 23 September 2025
Available online 24 September 2025

domain partitioning or kernel selection rules. It induces flexible covariance structures that adapt to localized features like stress concentrations or fracture zones [12], directly addressing the stationarity limitations of conventional GP. While deep neural networks (DNN) have demonstrated remarkable capabilities as deterministic function approximators in domains such as fractional integro-differential equations modeling [13], psychological disease analysis [14], and materials recycling systems [15], they fundamentally lack intrinsic probabilistic frameworks for uncertainty quantification, a critical requirement in reliability analysis characterized by scarce failure samples. DNN typically rely on post-hoc heuristics like dropout or ensembles that often underestimate epistemic uncertainty in data-scarce regimes. In contrast, DGP operates as Bayesian nonparametric models that explicitly propagate both aleatoric and epistemic uncertainties through posterior predictive distributions via exact inference techniques, while adaptively learning latent nonlinear mappings across hierarchical layers [16]. DGP employs shared hyperparameters across layers, and through the composition of multiple Gaussian processes, achieves higher-order nonlinear mappings of greater complexity. Crucially, DGP inherits the intrinsic uncertainty quantification properties of GP, yielding predictive confidence intervals that enhance model reliability. Through techniques such as sparse approximations and variational inference, DGP can be scaled to accommodate large datasets. Their hierarchical architecture further contributes to regularization, mitigating overfitting while enhancing generalization performance. Collectively, these capabilities enable DGP to effectively model complex patterns, establishing them as powerful tools for sophisticated datasets.

Consequently, DGP has garnered increasing attention from researchers worldwide. Extensive investigations have yielded refinements to DGP methodologies and expanded their application pathways. Regarding computational efficiency, Yu et al. [17] postulated a DGP based on implicit posterior variational inference to enhance inference speed and computational efficiency. Ding et al. [18] propose an efficient scheme for accurate inference and efficient training based on a range of Gaussian Processes, called the Tensor Markov Gaussian Processes, which significantly boosts the computational efficiency. In terms of predictive accuracy, Hebbal et al. [19] integrated DGP with non-parametric Bayesian mapping, establishing connections between input spaces of different fidelity levels through Gaussian processes, which achieved significant improvements in both predictive accuracy and uncertainty quantification. Liu et al. [20] proposed a Variational DGP framework employing an Adaptive Inter-layer Variational Inference strategy. This approach enhances information propagation and feature extraction, thereby improving prediction accuracy. Nakamura et al. [21] advocated for self-attentive Gaussian process layers to capture long-range dependencies in input sequences, enhancing performance in subjective auditory testing. In uncertainty quantification applications, Radaideh and Kozlowski [22] implemented DGP-based surrogates for uncertainty propagation in 91-dimensional nuclear data, quantifying uncertainties in high-fidelity simulations of nuclear reactor systems. For classification tasks, Fei et al. [23] developed an active learning methodology based on DGPs for binary and multi-class problems, combining DGP's nonlinear modeling capacity with strategic sample selection to reduce annotation costs and improve performance. Yang et al. [24] formulated a Bayesian active learning approach using DGP for choice models with correlated Gumbel noise, minimizing pairwise comparisons to efficiently learn individual preferences and identify optimal selections.

Early DGP frameworks, however, faced practical barriers in engineering contexts. Inferential complexity hindered deployment: variational approximations [25] compromised UQ fidelity, although this compromise can be mitigated by increasing samples [26], it introduces additional computational burden, while Markov Chain Monte Carlo (MCMC) samplers exhibited prohibitive computational costs for sequential design workflows [27]. A pivotal advance came from Sauer et al. [28], whose hybrid Gibbs-ESS-Metropolis algorithm enabled efficient full Bayesian inference for DGPs under limited simulation budgets. By leveraging Elliptical Slice Sampling (ESS) to traverse multimodal posterior distributions of latent layers and integrating active learning criteria for sequential design, their R package `deepgp` established the first computationally tractable DGP framework tailored to computer experiments.

While Sauer et al. established a robust foundation for DGP-based surrogate modeling, their practical impact in structural reliability analysis remains limited. This limited adoption stems from two critical barriers inherent in the R-based `deepgp` implementation: computational inefficiency and workflow incompatibility. First, R lacks capabilities for large-scale matrix operations. Second, structural reliability methodologies are predominantly implemented and shared in MAPLE [29] and MATLAB [30], forcing researchers to develop ad hoc cross-platform interfaces that compromise reproducibility.

Recognizing MATLAB's superior matrix computation efficiency, particularly for inverse covariance matrix operations essential in deep Gaussian processes, we have developed a MATLAB `deepgp` Toolbox that ports and extends the framework of Sauer et al. to the MATLAB ecosystem. Code fidelity during translation is rigorously ensured through validation protocols using Sobol or Halton sequences. Furthermore, the toolbox incorporates active learning methodologies for structural reliability analysis, demonstrating the implementation and effectiveness of the MATLAB-based `deepgp` in structural reliability applications through benchmark numerical examples and practical engineering case studies.

The remainder of this paper is structured as follows. Section 2 presents the theoretical foundations of, DGP, MCMC sampling, and active learning methodologies. Section 3 details the Toolbox implementation of DGP, including the architectural framework and representative code excerpts. In Section 4, we conduct computational benchmarking using identical sequences, comparing prediction accuracy and computational efficiency between R and MATLAB implementations. Section 5 demonstrates structural reliability applications of the DGP through two case studies. The concluding section synthesizes key contributions and suggests future research directions.

2. Theoretical background

To frame our contribution, we summarize key methodologies employed herein: Deep Gaussian Processes, MCMC sampling, and Active Learning.

2.1. Deep Gaussian Process

A DGP extends the hierarchical representation of standard GP by composing multiple layers of latent Gaussian mappings [11]. Formally, an L-layer DGP defines the output y through a chain of transformations:

$$\mathbf{y} = \tau^2[f_L(f_{L-1}(\cdots f_1(\mathbf{x})\cdots)) + \boldsymbol{\epsilon}], \boldsymbol{\epsilon} \sim \mathcal{N}(0, g\mathbf{I}) \tag{1}$$

where each latent function $f_l(\cdot)$ follows an independent GP prior:

$$f_l(\cdot) \sim \mathcal{GP}\big(0, K_{\theta_l}(\cdot)\big), l = 1, 2, ..., L \tag{2}$$

where $K_\theta^{ij}(\mathbf{x}) = \exp(-\| x_i - x_j\|^2/\theta)$.

In the isotropic Gaussian kernel, τ^2, θ, and g are hyperparameters governing the signal variance, length scale, and noise level, respectively.

This architecture induces non-stationary covariance structures by warping the input space through successive nonlinear mappings. The log-likelihood function for an L-layer DGP decomposes into conditional probabilities across hierarchical layers. Given observed data $\mathcal{D} = \{\mathbf{X}_n, \mathbf{Y}_n\}$, the joint log-likelihood integrates latent variables $\mathbf{F} = \{\mathbf{F}_1, ..., \mathbf{F}_L\}$:

$$\mathcal{L}(\mathbf{Y}_n|\mathbf{X}_n) = \int \mathcal{L}(\mathbf{Y}_n|\mathbf{F}_L) \prod_{l=2}^{L} \mathcal{L}(\mathbf{F}_l|\mathbf{F}_{l-1}) d\mathbf{F}, \mathbf{F}_1 \equiv \mathbf{X}_n \tag{3}$$

where $\mathcal{L}(\mathbf{Y}_n|\mathbf{F}_L)$ and $\mathcal{L}(\mathbf{F}_l|\mathbf{F}_{l-1})$ are defined in (4).

$$\log\mathcal{L}(\mathbf{Y}_n|\mathbf{X}_n) = -\frac{1}{2}\mathbf{Y}_n^T[\Sigma(\mathbf{X}_n)]^{-1}\mathbf{Y}_n - \frac{1}{2}\log|\Sigma(\mathbf{X}_n)| - \frac{n}{2}\log 2\pi \tag{4}$$

Under the reference prior $\pi(\tau^2) \propto 1/\tau^2$, the integrated marginal log-likelihood for any layer admits a closed form:

$$\begin{aligned} &\log\mathcal{L}(\mathbf{Y}_n|\mathbf{F}_L) \propto -\frac{n}{2}\log(n\widehat{\tau}^2) - \frac{1}{2}\log|\mathbf{K}_\theta(\mathbf{F}_L) + g\mathbf{I}_n| \\ &\widehat{\tau}^2 = \frac{1}{n}\left[\mathbf{Y}_n^T(\mathbf{K}_\theta(\mathbf{F}_L) + g\mathbf{I}_n)^{-1}\mathbf{Y}_n\right] \end{aligned} \tag{5}$$

In DGP architecture, each latent layer $\mathbf{F}_l (2 \le l \le L)$ may contain multiple nodes to enhance representational capacity, as shown in Fig. 1. For layer l, we define:

$$\mathbf{F}_l = \left[\mathbf{F}_{l,1}, \mathbf{F}_{l,2}, ..., \mathbf{F}_{l,r}\right] \in \mathbb{R}^{n \times rl} \tag{6}$$

Moreover, Sauer et al. empirically observed optimal model performance when $\dim(\mathbf{F}_l) = \dim(\mathbf{X}_n) = d$. In contrast to Damianou's support for latent layer latent node correlation, Sauer's team recommends structural simplification to alleviate computational burden, which includes (i) Conditional Independence, (ii) Isotropic Kernels.

$$\begin{aligned} &\text{(i)}\mathrm{Cov}\left(\mathbf{F}_{l,i}, \mathbf{F}_{l,j}|\mathbf{X}_n\right) = 0, i \ne j \\ &\text{(ii)}K_{\theta_{\mathbf{F}_{l,i}}}(\mathbf{X}_n) = \exp\left(-dist(\mathbf{X}_n)\Big/\theta_{\mathbf{F}_{l,i}}\right) \\ &K_{\theta_{\mathbf{Y}_n}}(\mathbf{X}_n) = \exp(-dist(\mathbf{X}_n)/\theta_{\mathbf{Y}_n}) \end{aligned} \tag{7}$$

Therefore, the log-likelihood for each latent layer is:

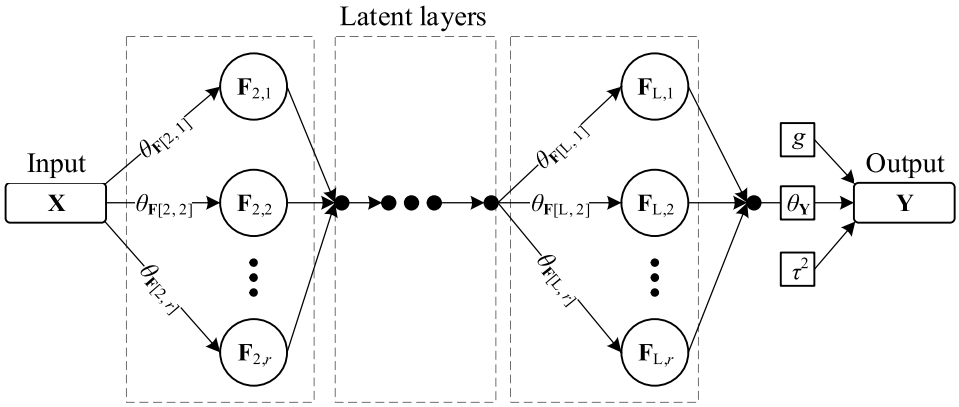


Fig. 1. Model structure for a L-layer DGP with r latent nodes.

$$\begin{aligned}\log\mathscr{L}(\mathbf{F}_l|\mathbf{F}_{l-1}) &\propto \sum_{i=1}^{r}\log\mathscr{L}\left(\mathbf{F}_{l,i}|\mathbf{F}_{l-1}\right)\\ &\propto \sum_{i=1}^{r}\left(-\frac{1}{2}\log\left|\mathbf{K}_{\theta_{\mathbf{F}_{l,i}}}(\mathbf{F}_{l-1})\right| - \frac{1}{2}\mathbf{F}_{l,i}^{T}\left[\mathbf{K}_{\theta_{\mathbf{F}_{l,i}}}(\mathbf{F}_{l-1})\right]^{-1}\mathbf{F}_{l,i}\right)\end{aligned} \tag{8}$$

At last, the complete log-likelihood is:

$$\log\mathscr{L}(\mathbf{Y}_n|\mathbf{X}_n) = \log\mathscr{L}(\mathbf{Y}_n|\mathbf{F}_l)\sum_{l=2}^{L}\log\mathscr{L}(\mathbf{F}_l|\mathbf{F}_{l-1}) \tag{9}$$

Bayesian inference targets the joint posterior:

$$p(\mathbf{F},\boldsymbol{\Theta}|\mathscr{D})\propto\mathscr{L}(\mathbf{Y}_n|\mathbf{F}_L)\prod_{l=2}^{L}\mathscr{L}(\mathbf{F}_l|\mathbf{F}_{l-1})\times\pi(\boldsymbol{\Theta}) \tag{10}$$

where $\pi(\boldsymbol{\Theta})$ is the prior distribution of hyperparameter $\boldsymbol{\Theta}=\{\theta, g\}$.

For test points $\mathbf{X}_p^*\in\mathbb{R}^{p\times d}$and latent layer predictions $\mathbf{F}_l^* = \left[\mathbf{F}_{l,1}^*,\mathbf{F}_{l,2}^*,...,\mathbf{F}_{l,rl}^*\right]\in\mathbb{R}^{p\times rl}$, combining (1) and (2) yields the predictive mean and covariance at each MCMC sample as:

$$\begin{aligned}&\text{for } l=1,...,L-1,\mathbf{F}_1^{(t)}\equiv\mathbf{X}_n,\mathbf{F}_1^{(t)*}\equiv\mathbf{X}_p^*\\ &\mu_{\mathbf{F}_{l+1,i}}^{(t)}\left(\mathbf{F}_l^{(t)*}\right) = K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)*},\mathbf{F}_l^{(t)}\right)\left[K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)}\right)\right]^{-1}\mathbf{F}_{l+1,i}^{(t)}\\ &\Sigma_{\mathbf{F}_{l+1,i}}^{(t)}\left(\mathbf{F}_l^{(t)*}\right) = K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)*}\right) - K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)*},\mathbf{F}_l^{(t)}\right)\left[K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)}\right)\right]^{-1}K_{\theta_{\mathbf{F}_{l,i}}^{(t)}}\left(\mathbf{F}_l^{(t)},\mathbf{F}_l^{(t)*}\right)\\ &\text{for } l=L\\ &\mu_{\mathbf{Y}}^{(t)}\left(\mathbf{F}_l^{(t)*}\right) = K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)*},\mathbf{F}_l^{(t)}\right)\left[K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)}\right)+g^{(t)}\mathbf{I}_n\right]^{-1}\mathbf{Y}_n\\ &\Sigma_{\mathbf{Y}}^{(t)}\left(\mathbf{F}_l^{(t)*}\right) = \hat{\tau}^{2(t)}\left\{K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)*}\right)+g^{(t)}\mathbf{I}_p - K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)*},\mathbf{F}_l^{(t)}\right)\left[K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)}\right)+g^{(t)}\mathbf{I}_n\right]^{-1}K_{\theta_{\mathbf{Y}}^{(t)}}\left(\mathbf{F}_l^{(t)},\mathbf{F}_l^{(t)*}\right)\right\}\end{aligned} \tag{11}$$

where t indexes samples across the MCMC chain.

By averaging over MCMC chains of length N, the predictive mean and covariance of the DGP are obtained as:

$$\begin{aligned}\mu_{\mathbf{Y}} &= \frac{1}{N}\sum_{t\in N}\mu_{\mathbf{Y}}^{(t)}\\ \Sigma_{\mathbf{Y}} &= \frac{1}{N}\sum_{t\in N}\Sigma_{\mathbf{Y}}^{(t)} + \frac{1}{N-p}\sum_{t\in N}\left(\mu_{\mathbf{Y}}^{(t)}-\mu_{\mathbf{Y}}\right)\left(\mu_{\mathbf{Y}}^{(t)}-\mu_{\mathbf{Y}}\right)^{T}\end{aligned} \tag{12}$$

Furthermore, practical implementation considers only diagonal elements of Σ to optimize computational resources while preserving uncertainty estimates.

2.2. MCMC sampling

2.2.1. Hyperparameter sampling

The Metropolis-Hastings (M-H) algorithm is a foundational MCMC method for generating samples from a target probability distribution $\pi(\mathbf{x})$, typically a Bayesian posterior. Its core idea is to construct a Markov chain that asymptotically converges to $\pi(\mathbf{x})$ via a proposal-accept mechanism:

For DGP, Gramacy and Lee [31] established a constrained proposal mechanism using a uniform sliding window when parameters require bounded support domains. Specifically, when sampling parameters are restricted to $\boldsymbol{\Theta}\in[\varepsilon,\infty)$, proposals are generated as:

$$\begin{aligned}&\boldsymbol{\Theta}\sim\text{Uniform}\left(\frac{l}{u}\boldsymbol{\Theta}^{(t-1)},\frac{u}{l}\boldsymbol{\Theta}^{(t-1)}\right)\\ &\frac{q(\boldsymbol{\Theta}^{(t-1)}|\boldsymbol{\Theta}^{*})}{q(\boldsymbol{\Theta}^{*}|\boldsymbol{\Theta}^{(t-1)})} = \frac{\boldsymbol{\Theta}^{(t-1)}}{\boldsymbol{\Theta}^{*}}\end{aligned} \tag{13}$$

Given a two-layer DGP ($\mathbf{X}\to\mathbf{F}_2\to\mathbf{Y}$), the acceptance probability α of hyperparameter $\theta_{\mathbf{Y}}$ combining (5) and (10) is given by:

$$\alpha = \min\left(1,\frac{\mathscr{L}\left(\mathbf{Y}_n|\mathbf{F}_2,\theta_{\mathbf{Y}}^{*},g\right)\pi\left(\theta_{\mathbf{Y}}^{*}\right)}{\mathscr{L}\left(\mathbf{Y}_n|\mathbf{F}_2,\theta_{\mathbf{Y}}^{(t-1)},g\right)\pi\left(\theta_{\mathbf{Y}}^{(t-1)}\right)}\cdot\frac{\theta_{\mathbf{Y}}^{(t-1)}}{\theta_{\mathbf{Y}}^{*}}\right) \tag{14}$$

2.2.2. Latent layer sampling

Elliptical Slice Sampling (ESS) is a robust MCMC technique designed for posterior distributions with multivariate Gaussian priors [32]. Given a target posterior distribution:

$$\pi(\mathbf{f}) \propto \mathscr{L}(\mathbf{f}) \times \mathcal{N}(\mathbf{f}; 0, \Sigma) \tag{15}$$

where $\mathscr{L}(\mathbf{f})$ is the likelihood function and $\mathcal{N}(\mathbf{f}; 0, \Sigma)$ is a zero-mean multivariate normal distribution prior, ESS iteratively generates samples through the following steps:

(i) Draw an auxiliary variable: $\mathbf{f}_{prior} \sim \mathcal{N}(0, \Sigma)$
(ii) Sample a random angle: $\gamma \sim \text{Uniform}(0, 2\pi)$
(iii) Set the angle bracket: $[\gamma_{min}, \gamma_{max}] = [\gamma - 2x, y]$

Then, generate a candidate point along an elliptical path, and compute the acceptance probability α:

$$\mathbf{f}^* = \mathbf{f}^{(t)} \cos\gamma + \mathbf{f}_{prior} \sin\gamma \tag{16}$$

$$\alpha = \min\left(1, \frac{\mathscr{L}(\mathbf{f}^*)}{\mathscr{L}(\mathbf{f}^{(t-1)})}\right) \tag{17}$$

Upon rejection, ESS shrinks the angle bracket by setting $\gamma_{min} = \gamma$ if $\gamma < 0$ and $\gamma_{max} = \gamma$ otherwise, then resamples $\gamma \sim \text{Uniform}(\gamma_{min}, \gamma_{max})$, generates a new proposal via (16), and repeats the acceptance probability evaluation (17) until either acceptance is achieved or a maximum iteration limit M is reached, distinguishing ESS from Metropolis-Hastings by avoiding value repetition while maintaining ergodicity.

ESS is adapted for sampling latent variables $\mathbf{F}_{l>1}$ in DGP by leveraging the hierarchical structure of DGP likelihoods. For each latent node in the DGP hierarchy, we have:

$$\mathbf{F}_{l,i}^{prior} \sim N\left(0, K_{\theta_{\mathbf{F}_{l,i}}}(\mathbf{F}_{l-1})\right) \tag{18}$$

$$\mathbf{F}_{l,i}^{*} = \mathbf{F}_{l,i}^{(t)} \cos\gamma + \mathbf{F}_{l,i}^{prior} \sin\gamma \tag{19}$$

$$\alpha = \min\left(1, \frac{\mathscr{L}\left(\mathbf{F}_l | \mathbf{F}_{l-1,i}^{*}, \mathbf{F}_{l-1,>i}^{(t-1)}, \mathbf{F}_{l-1,<i}^{(t)}\right)}{\mathscr{L}\left(\mathbf{F}_l | \mathbf{F}_{l-1,i}^{(t-1)}, \mathbf{F}_{l-1,>i}^{(t-1)}, \mathbf{F}_{l-1,<i}^{(t)}\right)}\right) \tag{20}$$

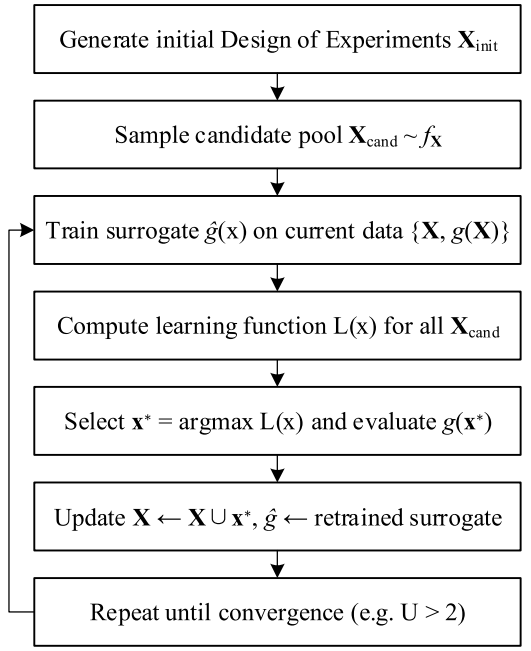


Fig. 2. AK-MCS framework.

where $\mathbf{F}_{l-1,>i}^{(t-1)}$ represents nodes with an index greater than i (updated in the current Gibbs cycle), and $\mathbf{F}_{l-1,<i}^{(t)}$ represents nodes with an index less than i (updated).

2.3. Active learning in reliability analysis

Active learning (AL) aims to reduce computational costs in reliability analysis by strategically selecting simulations that maximize information gain about failure regions. For a limit-state function $g(\mathbf{x})$ defining failure as $g(\mathbf{x}) < 0$, AL iteratively refines a surrogate model using adaptively chosen samples. Active Kriging Monte Carlo Simulation (AK-MCS) was proposed by Echard et al. [33], AK-MCS integrates Kriging surrogates with MCS for efficient reliability assessment, as shown in Fig. 2.

Several classical learning functions and their corresponding convergence criteria are introduced below:

$$U(\mathbf{x}) = |\boldsymbol{\xi}|, \min U(\mathbf{x}) > 2 \tag{21}$$

$$\begin{aligned} &EFF(\mathbf{x}) = \mu_{\hat{g}}(\mathbf{x})[2\Phi(-\boldsymbol{\xi}) - \Phi(-2-\boldsymbol{\xi}) - \Phi(2-\boldsymbol{\xi})] \\ &-\sigma_{\hat{g}}(\mathbf{x})[2\phi(-\boldsymbol{\xi}) - \phi(-2-\boldsymbol{\xi}) - \phi(2-\boldsymbol{\xi})] \\ &+2\sigma_{\hat{g}}(\mathbf{x})[\Phi(2-\boldsymbol{\xi}) - \Phi(-2-\boldsymbol{\xi})] \\ &\max EFF(\mathbf{x}) < 0.01 \text{or} 0.001 \end{aligned} \tag{22}$$

$$\begin{aligned} H(\mathbf{x}) = &\left| \ln\left(\sqrt{2\pi}\sigma_{\hat{g}}(\mathbf{x}) + \frac{1}{2}\right)[\Phi(2-\boldsymbol{\xi}) - \Phi(-2-\boldsymbol{\xi})] \right. \\ &\left. -\left[\frac{2\sigma_{\hat{g}}(\mathbf{x}) - \mu_{\hat{g}}(\mathbf{x})}{2}\phi(2-\boldsymbol{\xi}) + \frac{2\sigma_{\hat{g}}(\mathbf{x}) + \mu_{\hat{g}}(\mathbf{x})}{2}\phi(-2-\boldsymbol{\xi})\right] \right| \\ &\max H(\mathbf{x}) < 0.5 \end{aligned} \tag{23}$$

where $\boldsymbol{\xi} = \mu_{\hat{g}}(\mathbf{x})/\sigma_{\hat{g}}(\mathbf{x})$, ϕ is the probability density function of the standard normal distribution, and Φ is its cumulative distribution function.

Upon satisfying the convergence criteria, the reliability and coefficient of variation (COV) can be computed as:

$$\begin{aligned} &R = 1 - P_f = 1 - \Pr\{\hat{g}(\mathbf{x}) < 0\} \approx 1 - \frac{1}{N}\sum_{i=1}^{N} I[\hat{g}(\mathbf{x})] \\ &I(x) = \begin{cases} 1, x \le 0 \\ 0, x > 0 \end{cases} \\ &COV_{pf} = \sqrt{\frac{1-P_f}{P_f \times N}} \end{aligned} \tag{24}$$

where N denotes the number of Monte Carlo samples.

3. Toolbox implementation

Although the theoretical framework in preceding sections accommodates arbitrarily deep DGP architectures, and Damianou &

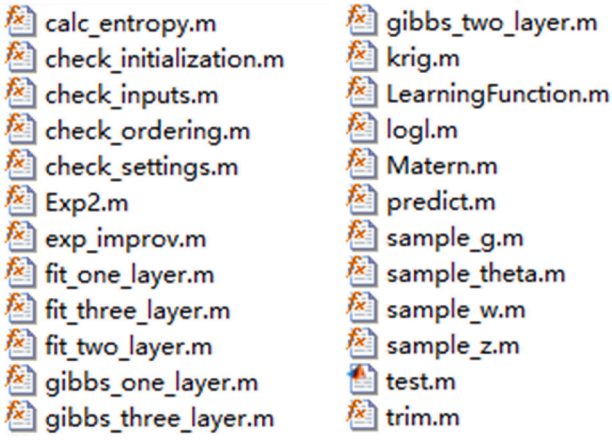


Fig. 3. `deepgp` toolbox contents.

Lawrence [11] employed a five-layer DGP for classification tasks, Radaideh & Kozlowski [22] demonstrate that two- or three-layer DGP are sufficiently expressive for surrogate modeling applications. Consequently, the current `deepgp` toolbox implementation specifically supports one-to-three-layer configurations using the streamlined notational convention $\mathbf{X}\rightarrow\mathbf{Z}\rightarrow\mathbf{W}\rightarrow\mathbf{Y}$. This notation supersedes generic layer indexing (e.g., $\mathbf{X}\rightarrow\mathbf{F}_2\rightarrow\mathbf{F}_3\rightarrow\mathbf{Y}$) to enhance code readability and implementation simplicity, while preserving identical mathematical representation. It must be emphasized that extending to additional layers is computationally feasible, owing to the modular code structure as discussed later.

The `deepgp` toolbox mainly consists of six modules, as shown in Fig. 3 and Table 1, namely Model Construction, Model Prediction, Gibbs Sampling Framework, Model Initialization and Validation, Kernel Function, and Active Learning. This section will briefly describe the capabilities of each module; detailed explanations are provided in Appendix A.

3.1. Model construction

This section describes the `fit_*_layer` functions for DGP model fitting and discusses the chain-pruning methodology implemented in `trim` for MCMC memory optimization.

The function interface employs MATLAB's name-value pair paradigm, where mandatory input arrays x and y are followed by optional parameter specifications. The arguments block enforces:

(i) Type safety via data type constraints,
(ii) Value domain validation,
(iii) Default initialization when parameters are omitted.

This design enables both minimal invocation and fine-grained configuration, as shown below.

fit = fit_one_layer(x, y)

fit = fit_two_layer(x, y)

fit = fit_three_layer(x, y)

Table 1
`deepgp` toolbox contents.

3.1 Model Construction	
`fit_one_layer`	Fit the L-layer DGP model to a given set of design data and given settings.
`fit_two_layer`	
`fit_three_layer`	
`trim`	Remove the specified number of MCMC iterations (starting at the first iteration). After these samples are removed, the remaining samples are optionally thinned.
3.2 Model Prediction	
`predict`	Use the DGP model to predict the function at one or more untried sites.
`krig`	Calculate conditional predictions and uncertainties for test points based on observed data using either Matern or exponential-squared kernels
3.3 Gibbs Sampling Framework	
`gibbs_one_layer`	Implement depth-specific Bayesian inference via MCMC, sampling covariance parameters for single-layer GP and jointly sampling hyperparameters/latent variables across hierarchical layers in DGP.
`gibbs_two_layer`	
`gibbs_three_layer`	
`sample_g`	Perform Metropolis-Hastings sampling for the DGP noise level g and length scale θ using a uniform sliding window proposal.
`sample_theta`	
`sample_w`	Perform Elliptical Slice Sampling of latent variables w and z in DGP, and update states.
`sample_z`	
`logl`	Calculate the log-likelihood function with Matern or Exp2 kernels, while deriving the conditional maximum likelihood estimate of τ^2.
3.4 Model Initialization and Validation	
`check_inputs`	Validate input data for Gaussian Process regression, and perform essential checks on input data dimensions, ranges, and normalization
`check_ordering`	Validate ordering vector for sampling sequences, and ensure ordering vector meets requirements for data indexing in MCMC sampling.
`check_initialized`	Validate and complete the initial parameter structure, and ensure all required fields exist, dimensions match requirements, and fill missing values for hidden layer matrices using kriging interpolation.
`check_settings`	Validate and complete hyperparameter settings for Gibbs sampling, ensure all required hyperparameters exist, and set appropriate defaults based on model layers, monowarp usage, and noise mode.
3.5 Kernel Function	
`Exp2`	Calculate the Gaussian kernel covariance function for Gaussian Process regression, including optional diagonal noise.
`Matern`	Calculate the Matern family of covariance functions for Gaussian processes, including noise terms for auto-covariance matrices. Three common smoothness levels are supported.
3.6 Active Learning	
`exp_improv`	Calculate the expected improvement over the current best function value on Gaussian predictive distributions.
`calc_entropy`	Calculate the information entropy of binary outcomes (pass/fail) based on Gaussian predictive distributions and a specified decision threshold
`LearningFunction`	Calculate the U/EFF/H-function acquisition metric to identify the most informative candidate point for failure-boundary refinement, accelerating reliability convergence with minimal simulation calls.

fit = fit_one_layer(x, y, 'nmcmc', nmcmc, 'normalize', true)

where x is an $n \times d$ matrix of input features, y is an n-dimensional vector of target outputs, with n and d representing the number of samples and input dimensions, respectively.

The trim function implements MCMC output postprocessing through burn-in removal and thinning, reducing autocorrelation while preserving the stationary distribution approximation, thereby optimizing storage efficiency and improving subsequent statistical inferences.

fit = trim(fit, burn, thin)

The trim function processes an estimated DGP structure (fit), requiring two parameters: burn specifies the initial MCMC iterations discarded (constrained as burn < total samples), while thin defines the thinning interval between retained samples (default: 1 = no thinning).

3.2. Model prediction

This section introduces computational routines for predictive inference in DGP models, centered on the primary function predict that generates posterior mean and variance estimates. This procedure fundamentally relies on krig, which implements Bayesian conditional expectation via Gaussian process conditioning on latent hierarchical representations.

Consistent with the fit_*_layer function interface, predict employs MATLAB's name-value syntax. It accepts the fitted model and test inputs as mandatory arguments, returning an augmented structure with three key additions: predictive means, predictive variances (or full covariance matrices), and the computational time required.

fit = predict(fit, xnew);

where fit denotes a pre-estimated DGP structure from fit_*_layer, and xnew represents test points, returning an augmented structure fit with predictive posterior summaries.

3.3. Gibbs sampling framework

The Gibbs Sampling Framework comprises three modular tiers. The gibbs_*_layer routines cyclically sample hyperparameters and latent states within corresponding fit_*_layer estimation workflows. The sample_* functions implement core MCMC transition kernels, invoked internally by gibbs_*_layer. The logl computes log-likelihood evaluations during sampling, providing the probabilistic backbone for M-H acceptance steps.

results = gibbs_one_layer(x, y, nmcmc, verb, initial, true_g, settings, v)

results = gibbs_two_layer(x, y, nmcmc, D, verb, initial, true_g, settings, v)

results = gibbs_three_layer(x, y, nmcmc, D, verb, initial, true_g, settings, v)

The gibbs_*_layer functions require eight mandatory inputs: the observed data matrix x and response vector y, MCMC chain length nmcmc, display flag verb, initialization structure initial containing starting parameter values, true noise precision true_g, configuration structure settings, and the Matern smoothness parameter ν. For multi-layer architectures, the latent dimensionality D constitutes an additional ninth required argument governing hierarchical composition. The function returns a structure, containing the sampling noise level g, the length scale θ, the latent layers, and the output scale τ^2, the log-likelihood trace, which is propagated to the final object fit.

results = sample_g(out_vec, in_dmat, g_t, theta, alpha, beta, l, u, ll_prev, v)

results = sample_theta(out_vec, in_dmat, g, theta_t, alpha, beta, l, u, outer, ll_prev, v, calc_tau2)

results = sample_w(out_vec, w_t, w_t_dmat, in_dmat, g, theta_y, theta_w, ll_prev, v)

results = sample_z(out_mat, z_t, z_t_dmat, in_dmat, g, theta_w, theta_z, ll_prev, v)

The sample_* functions accept: out_vec as the response vector y, out_mat as the latent response matrix w, in_dmat denoting the precomputed pairwise Euclidean squared distance matrix, shape-rate parameterization alpha and beta for gamma priors, l and u defining the uniform sliding window proposal boundaries, ll_prev as the previous log-likelihood evaluation, and the flag calc_tau2 controlling output scale τ^2 estimation. These functions return a sampled parameter structure, which is propagated to the parent gibbs_*_layer routine for hierarchical updating.

The framework's computational components automatically populate input parameters and manage output propagation through standardized interfaces, with Table 2 providing implementation-specific pseudo-code for the cyclic sampling mechanics governing hyperparameter updates and latent state transitions within hierarchical layers.

3.4. Active learning

This section formalizes acquisition functions for sequential experimental design in reliability engineering, facilitating informed sampling through Gaussian predictive distributions: `exp_improv` quantifies potential gains beyond current optima, `calc_entropy` evaluate information content of binary outcomes against specified decision thresholds, and `LearningFunction` synthesize U-, EFF-, and H-metrics to pinpoint maximally informative candidates for failure-boundary refinement, accelerating reliability convergence with minimal simulation expenditure.

The learning function invocation focuses on reliability-oriented active learning:

[L, index] = LearningFunction('u', mu, sig)

where '`u`' specifies the uncertainty sampling function, `mu` and `sig` represent predictive mean and standard deviation, with output `L` returning the evaluation metric per candidate point and `index` indicating the optimal design point's position within the sample pool.

4. Performance comparison of R and MATLAB

The `deepgp` implementation in R relies heavily on computationally intensive matrix operations, particularly within its MCMC framework for hyperparameter and latent layer estimation. This dependence inherently poses a significant scalability challenge: as the Markov chain length increases or the size of training or prediction datasets grows, the execution time becomes prohibitively long.

While the R version offers multi-core acceleration options, the improvement often remains insufficient for practical applications requiring rapid iteration or handling larger datasets. Recognizing these limitations and capitalizing on MATLAB's renowned efficiency and optimized libraries for matrix manipulation, we undertook the re-implementation of `deepgp` within the MATLAB environment. This endeavor included incorporating a standardized preprocessing pipeline and establishing a modularized framework to ensure compatibility with diverse learning functions. Following the foundational theoretical exposition of Deep Gaussian Processes and the architectural overview of our MATLAB `deepgp` Toolbox presented in the preceding sections, this section presents a comparative analysis of the two platform implementations.

The core objectives are twofold: firstly, to rigorously validate that our MATLAB implementation achieves predictive fidelity consistent with the established R reference, thereby ensuring model integrity during translation; and secondly, to quantitatively evaluate and compare the computational efficiency gains afforded by the MATLAB platform. All computations were performed on a workstation equipped with an Intel(R) Core (TM) i7-7700 K CPU @ 4.20 GHz, 16.0 GB of RAM, and an NVIDIA GeForce GTX 750 Ti GPU. The simulations were implemented in MATLAB R2021b and R version 4.4.3.

Table 2
The pseudo-code for sampling (three-layer DGP).

gibbs_three_layer
compute distance matrix **dist(x)**, **dist(w)**, **dist(z)**
initialize .., $\theta_y^{(1)}$, $\theta_z^{(1)}$, $\theta_w^{(1)}$, $\mathbf{W}^{(1)}$, $\mathbf{Z}^{(1)}$
for $t = 2$ to nmcmc
sample_g
$g^{(t)} \sim \pi\left(g\middle\vert\mathbf{Y}_n, \mathbf{W}^{(t-1)}, \theta_y^{(t-1)}, g^{(t-1)}\right)$, M-H (14) via $\mathscr{L}\left(\mathbf{Y}_n\middle\vert\mathbf{W}, \theta_y, g\right)$.
sample_theta
$\theta_y^{(t)} \sim \pi\left(\theta_y\middle\vert\mathbf{Y}_n, \mathbf{W}^{(t-1)}, \theta_y^{(t-1)}, g^{(t)}\right)$, M-H (14) via $\mathscr{L}\left(\mathbf{Y}_n\middle\vert\mathbf{W}, \theta_y, g\right)$.
for $i = 1$ to d
sample_theta
$\theta_{w_i}^{(t)} \sim \pi\left(\theta_{w_i}\middle\vert\mathbf{W}_i^{(t-1)}, \mathbf{Z}^{(t-1)}, \theta_{w_i}^{(t-1)}\right)$, M-H (14) via $\mathscr{L}(\mathbf{W}_i\vert\mathbf{Z}, \theta_{w_i})$.
insert additional sample_theta calls for each additional latent layer.
sample_theta
$\theta_{z_i}^{(t)} \sim \pi\left(\theta_{z_i}\middle\vert\mathbf{Z}_i^{(t-1)}, \mathbf{X}_n, \theta_{z_i}^{(t-1)}\right)$, M-H (14) via $\mathscr{L}(\mathbf{Z}_i\vert\mathbf{X}_n, \theta_{z_i})$.
end
for $i = 1$ to d
sample_w
$\mathbf{W}_i^{(t)} \sim \pi\left(\mathbf{W}\middle\vert\mathbf{Y}_n, \mathbf{W}_{\geq i}^{(t-1)}, \mathbf{W}_{\leq i}^{(t)}, \mathbf{Z}^{(t-1)}, g^{(t)}, \theta_y^{(t)}, \theta_w^{(t)}\right)$, ESS via (20).
insert additional sample_Fl calls for each additional latent layer, synchronously.
sample_z
$\mathbf{Z}_i^{(t)} \sim \pi\left(\mathbf{Z}\middle\vert\mathbf{X}_n, \mathbf{W}^{(t)}, \mathbf{Z}_{\geq i}^{(t-1)}, \mathbf{Z}_{\leq i}^{(t)}, \theta_w^{(t)}, \theta_z^{(t)}\right)$, ESS via (20).
end
end
Return results

4.1. Ensuring consistency

Given that the random number generation mechanisms in R and MATLAB differ significantly, directly seeding might not guarantee identical sampling sequences and thus could compromise the consistency of parameter estimates across platforms. To ensure deterministic and reproducible results when fitting models to the same dataset, this study utilizes fixed segments of low-discrepancy sequences (specifically Sobol and Halton sequences), identical for both platforms, to derive the requisite pseudo-random numbers. For demonstration, Fig. 4 depicts the estimation procedure applied to a three-layer DGP model, and Table B.3 shows relevant pseudo-codes implementing this approach. The pseudo-code in Table B.3 is implemented in both the MATLAB and R platforms. Though the specific implementation details differ, the core algorithmic logic remains functionally identical across platforms.

Model result consistency is validated using the two examples below, testing both one-, two-, and three-layer DGP models. The corresponding MATLAB and R code implementations for this procedure are provided in Table B.4.

$$f_1(x) = \begin{cases} 1.35\cos(12\pi x), x \in [0, 0.33) \\ 1.35, x \in [0.33, 0.66) \\ 1.35\cos(6\pi x), x \in [0.66, 1] \end{cases} \tag{25}$$

$$f_2(x) = 10x_1\exp\left(-x_1^2 - x_2^2\right), x_1, x_2 \in [-2, 4] \tag{26}$$

The comparative prediction results of $f_1(x)$ across the two platforms (MATLAB and R) are summarized in Table 3 below. For the one-layer DGP model, observed discrepancies were below 10^{-15}. The two-layer model exhibited discrepancies smaller than 10^{-8}, while the three-layer model demonstrated discrepancies below 10^{-7}. To further visualize these inter-platform differences, the base-10 logarithm of the absolute prediction discrepancies across all ten test points for each of the three model depths is plotted in Fig. 5.

4.2. Evaluating computational efficiency

Having established the functional consistency and predictive fidelity of the MATLAB `deepgp` implementation relative to the R reference in the preceding subsection, this section focuses on quantitatively assessing their comparative computational performance. Specifically, we benchmark the execution time required for model fitting and prediction across both platforms under systematically increasing computational loads.

To achieve this, we conduct controlled experiments where the size of both the training dataset (used for fitting) and the testing dataset (used for prediction) is progressively scaled upward. The core objective is to measure and analyze the resulting computational time expenditure on each platform independently, isolating runtime as the key metric of interest.

To evaluate computational efficiency, we systematically varied the training sample size $n = \{10, 20, 50, 100, 200\}$ and the prediction sample size $p = \{1\text{ k}, 2\text{ k}, 5\text{ k}, 10\text{ k}, 20\text{ k}\}$. The length of the MCMC sampling chain was fixed at 1000 iterations for all experiments.

While progressively increasing the chain length might theoretically provide deeper comparison, both platform implementations rely on computationally analogous looping mechanisms for MCMC, limiting its utility in revealing core platform-specific computa-

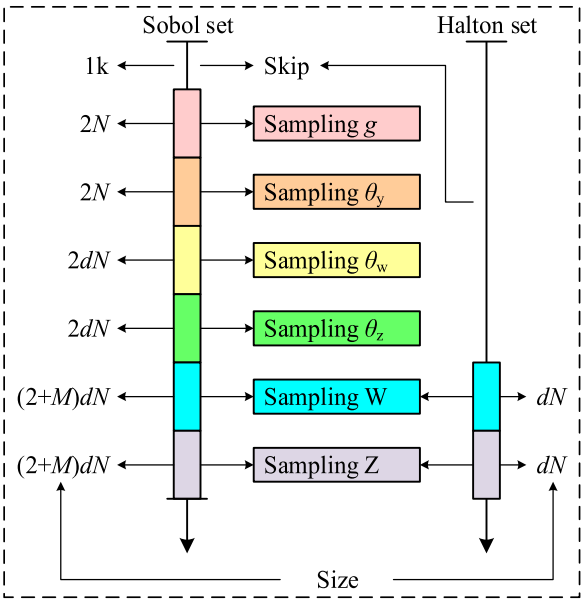


Fig. 4. Controlled low-discrepancy sequence implementation for cross-platform consistent estimation.

Table 3
Prediction discrepancies of $f_1(x)$.

Platform	**MATLAB (R)**	
Function	mean	variance
fit_one_layer	0.68070699175524 (0.68070699175524)	0.47437422913253 (0.47437422913254)
	-0.41924847814065 (-0.41924847814066)	0.33245977850300 (0.33245977850301)
	1.19233915447196 (1.19233915447196)	0.33791170656551 (0.33791170656551)
	1.24444582726399 (1.24444582726399)	0.33685527666199 (0.33685527666199)
	-0.76765440147622 (-0.76765440147622)	0.40426330093783 (0.40426330093783)
fit_two_layer	0.64062474419909 (0.64062474419493)	0.58234532322146 (0.58234532321259)
	-1.22454272943120 (-1.22454272943695)	0.30355231280927 (0.30355231280691)
	1.35031574033132 (1.35031573840058)	0.00004759725686 (0.00004759866622)
	1.35471494419793 (1.35471494266751)	0.00024703825287 (0.00024703924597)
	-0.62253896737648 (-0.62253896739753)	0.53017684234658 (0.53017684233032)
fit_three_layer	0.59632224573445 (0.59632223770470)	0.25942831006360 (0.25942832386693)
	-0.42274800541723 (-0.42274791558482)	0.72133114883809 (0.72133122276743)
	1.38029972536263 (1.38029973449570)	0.01868739694111 (0.01868740301406)
	1.41009967016150 (1.41009967440552)	0.05568198386631 (0.05568197704449)
	-0.84708931702962 (-0.84708938417351)	0.16925334957701 (0.16925340378135)

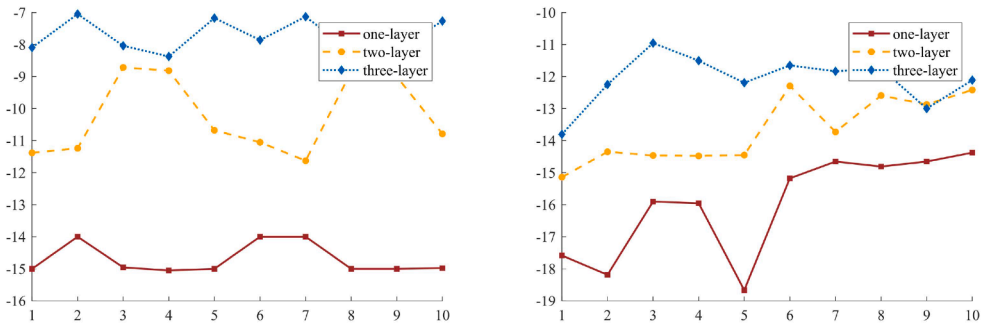


Fig. 5. Log-absolute discrepancies visualization of $f_1(x)$ (Left) and $f_2(x)$ (Right).

tional advantages while significantly increasing experimental complexity. Execution times for model fitting and subsequent prediction were measured independently for each combination of n and p across all three model depths, with prediction points generated by the following five-dimensional function.

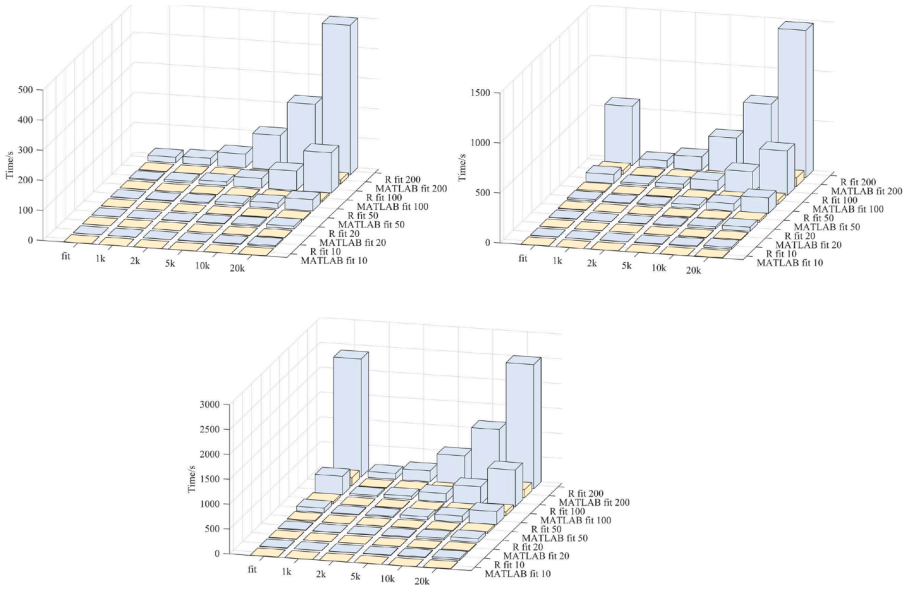


Fig. 6. Computational efficiency comparison one-, two-, three-layer DGP.

$$f_3(x) = \sqrt{\frac{12}{5}}\left(\sum_{i=1}^{5} x_i - 2.5\right) \tag{27}$$

The MATLAB implementation realizing this computational process is documented in Table B.5 (with algorithmically equivalent R code). Resultant data aggregates are compiled in Table B.6 to Table B.8, while timing comparisons are visualized in Fig. 6.

When utilizing the `fit_one_layer` function, the MATLAB `deepgp` implementation demonstrated computational efficiency gains of approximately 12 to 37 times compared to the R counterpart for combined fitting and prediction. For multi-layer models, this speedup factor stabilizes at roughly 15-fold.

5. Case studies

Leveraging the MATLAB `deepgp` toolbox developed in section 3, we implement an active learning methodology termed Active learning reliability method combining Deep Gaussian Process and Monte Carlo Simulation (AL-DGP-MCS), synthesizing DGP surrogates with the AK-MCS framework for efficient reliability analysis. For each case, failure probabilities are computed using AL-DGP-MCS and benchmarked against MCS results serving as the ground truth.

Quantitative evaluations focus on three critical performance metrics, the cumulative number of training samples required for convergence, relative error in failure probability estimates and the coefficient of variation in (24) characterizing the stability of probabilistic predictions. These analyses collectively validate AL-DGP-MCS's capacity to balance computational efficiency with precision in complex reliability scenarios.

5.1. Numerical example

Consider a two-dimensional system [34] characterized by a small system failure probability with two distinct failure modes in a series configuration. The performance function for each failure mode is defined as follows:

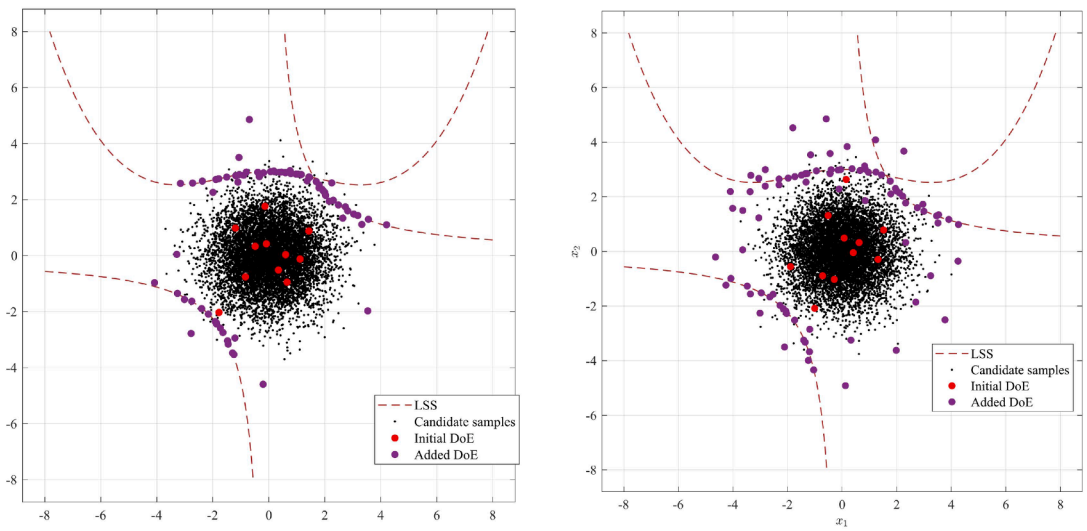
$$g_1(\mathbf{x}) = \begin{cases} b - 1 - x_2 + \exp\left(-x_1^2/2\right) + \left(x_1/5\right)^4 \\ b^2/2 - x_1 x_2 \end{cases}, b = 3 \tag{28}$$

where x_1 and x_2 are two standard normal distribution random variables.

Fig. 7 illustrates the u-function learning process for AL-DGP2-MCS (two-layer DGP) and AK-MCS in a numerical case study, where AL-DGP2-MCS began with 11 initial DOE points and incorporated 71 additional sample points, while AK-MCS started with 11 initial DOE points and added 83 more points.

Notably, the majority of the added points for AL-DGP2-MCS are located on the limit state surface, demonstrating superior performance over AK-MCS and confirming its robust learning capability.

To further demonstrate the capabilities of the `deepgp`, reliability analyses were conducted using DGP1, DGP2, and a conventional Kriging model through 15 independent runs each. Fig. 8 presents boxplots of the failure probability estimate across these trials and



(a) AL-DGP2-MCS+U (b) AK-MCS+U

Fig. 7. Active learning sampling evolution on the limit state surface.

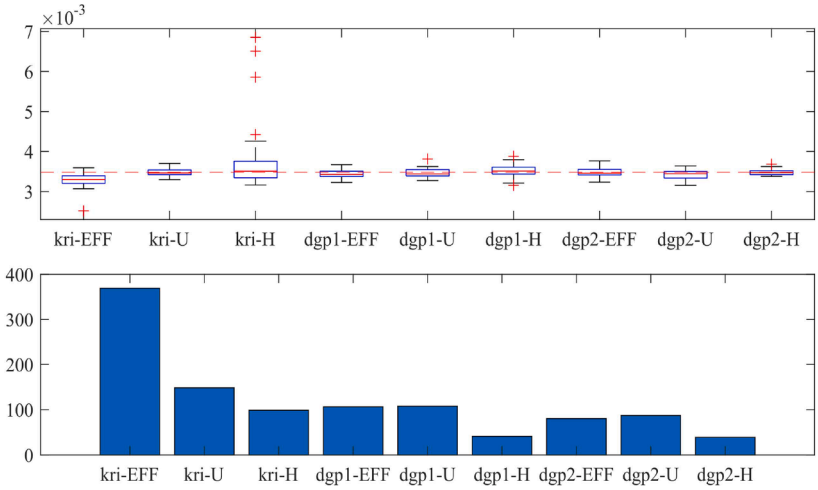


Fig. 8. Boxplots of all compared methods (Numerical example).

quantifies the corresponding number of samples added during active learning. Both DGP1 and DGP2 consistently required fewer added samples than the Kriging model across all replicates, confirming enhanced sampling efficiency.

The calculated system failure probabilities are summarized in Table 4, where P_f denotes the estimated system failure probability, N_{call} represents the number of true system response evaluations (equivalent to both the model fitting sample size and iteration count), ε indicates the relative error against the MCS benchmark, and COV designates the coefficient of variation. To mitigate stochastic uncertainty, mean values were computed from thirty independent runs for each methodology.

The two-layer DGP model achieves substantial modeling accuracy and predictive stability, quantified via evaluation errors and coefficients of variation, while requiring significantly fewer sampling points than conventional Kriging. Though methods proposed in [34] and [35] demonstrate superior computational efficiency, their predictive precision (in Table 1 of [34]) remains inferior to the DGP framework. This comparative advantage in reliability-oriented performance metrics validates DGP's potential for structural reliability applications where computational resource constraints necessitate sample-efficient surrogate modeling without compromising uncertainty quantification fidelity.

5.2. *Hydraulic excavator*

Practical engineering reliability problems are inherently complex, often involving more than 10 design variables and lacking explicit limit-state functions. Finite element simulations for such problems are highly time-consuming, requiring tens of minutes to several hours per evaluation, which severely limits the number of feasible function evaluations.

Fig. 9 illustrates the 3D model of a hydraulic excavator's attachment (ZE215G, supported by the ZOOMLION under project funding). Even in a simplified form, this assembly comprises 132 individual components. Existing surrogate models face significant challenges under these constraints, which motivated the development of our toolbox and its extension to reliability analysis.

The hydraulic excavator operates with its main body fixed, utilizing the boom, arm, and bucket cylinders to provide digging force while resisting ground reaction forces and gravity. To streamline the computational model, fixed constraints are applied at the boom hinge point and bucket; the cylinders are treated as rigid bodies and replaced with equivalent thrust forces (F_{boom}, F_{arm}, and F_{bucket}) derived from sensor-monitored data, as illustrated in Fig. 9. For more data details, please refer to [36].

Table 4
Results of the numerical case.

Method	N_{call}	P_f ($\times 10^{-3}$)	ε %	COV
MCS	10^6	3.483	-	-
AK-MCS+EFF	369.3	3.285	5.685	0.040
AK-MCS+U	148.3	3.463	0.574	0.047
AK-MCS+H	98.6	3.834	10.078	0.047
AL-DGP1-MCS+EFF	106.4	3.438	1.292	0.048
AL-DGP1-MCS+U	107.6	3.465	0.517	0.047
AL-DGP1-MCS+H	41.1	3.522	1.120	0.047
AL-DGP2-MCS+EFF	80.3	3.495	0.345	0.047
AL-DGP2-MCS+U	87.0	3.465	0.517	0.046
AL-DGP2-MCS+H	42.7	3.486	0.086	0.033
Xiao et al., Table 1 in [34]	27.4	3.454	0.833	-
AK-SYS, Table 1 in [34]	33.1	3.469	0.402	-

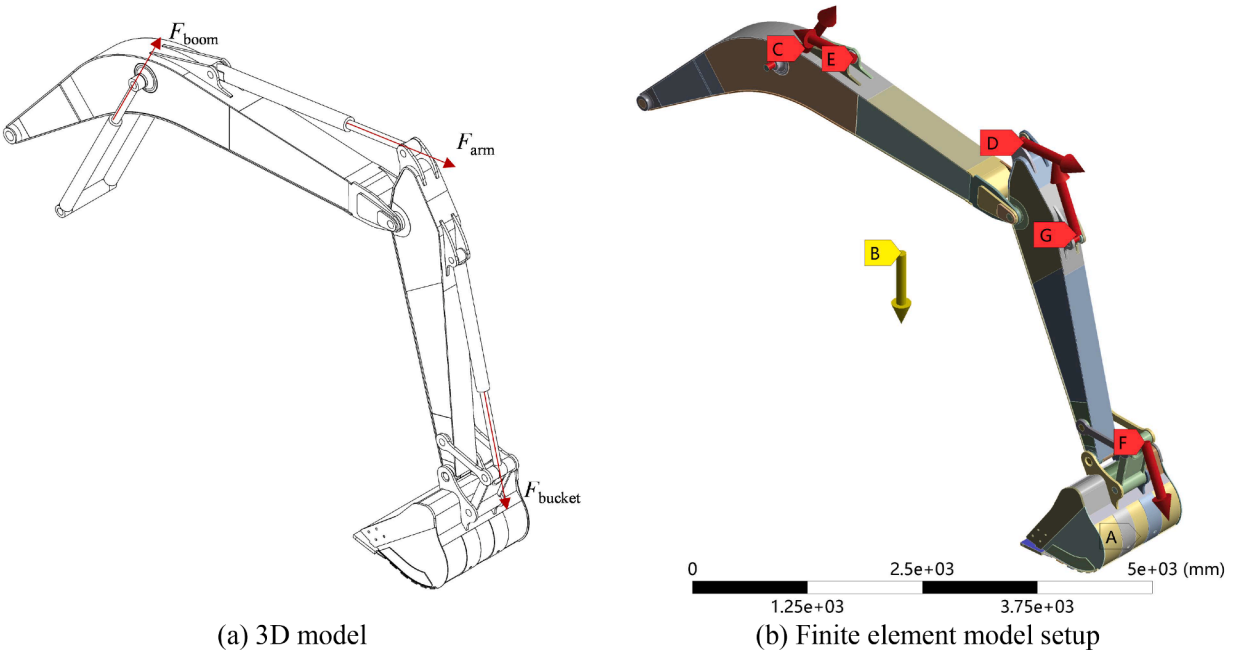


Fig. 9. Hydraulic excavator attachment.

As illustrated in Fig. 10, a global mesh size of 500 mm was adopted, with local refinements of 15 mm for stiffeners, 20 mm for spacers, and 30 mm for lugs.

Fig. 11 illustrates the stress and strain contour plots from a finite element analysis under random parameters. The maximum stress and strain concentrations are located at the pin joint connecting the boom and arm components. The limit state is defined as a dual-constraint system where the maximum stress must not exceed the allowable stress [σ] and the maximum strain must remain below the allowable strain [ε]. Therefore, two limit state functions, g_{stress} and g_{strain}, are defined as follows:

$$\begin{aligned} g_{stress} &= [\sigma] - \sigma_{max}(\mathbf{x}) \\ g_{strain} &= [\varepsilon] - \varepsilon_{max}(\mathbf{x}) \end{aligned} \tag{29}$$

where $\sigma_{max}(\mathbf{x})$ and $\varepsilon_{max}(\mathbf{x})$ represent the maximum simulated stress and strain, respectively, computed via finite element analysis based on the input parameters $\mathbf{x} = [F_{boom}, F_{arm}, F_{bucket}, d_1, d_2, d_3, d_4, d_5]$. And $[\sigma] = 295$ MPa, $[\varepsilon] = 0.0014$.

Table B.9 in Appendix B presents simulation data utilized for initial surrogate model construction, where the first eight columns of input parameters are processed through finite element simulations to compute the resulting maximum stress σ_{max} and maximum strain ε_{max} values documented in the ninth and tenth columns, respectively.

Based on (29), The probability of failure is calculated as:

$$\begin{aligned} P_f &= \Pr(g_{stress} < 0 \cup g_{strain} < 0) \\ &= \Pr(\min(g_{stress}, g_{strain}) < 0) \end{aligned} \tag{30}$$

Then, the limit state function for the excavator system is defined as follows:

$$\begin{aligned} g_2 &= \min(g_{stress}, g_{strain}) \\ &= \min([\sigma] - \sigma_{max}(\mathbf{x}), [\varepsilon] - \varepsilon_{max}(\mathbf{x})) \end{aligned} \tag{31}$$

To better enable parametric simulation under uncertainty, we established a co-simulation platform integrating numerical analysis software, three-dimensional modeling software, and finite element analysis software, building on our prior work [36]. Key uncertain parameters include both structural dimensions (thicknesses of plates d) and operational loads. These variables were modeled with normal distributions to quantify uncertainties, detailed in Table 5. The mean values of the structural parameters correspond to the design dimensions of the excavator, while their standard deviations reflect manufacturing tolerances. The mean and standard deviation of the load parameters were derived from sensor data and approximated using a normal distribution.

Fig. 12 and Table 6 present comparative analysis results for this case study. Given that a single finite element simulation requires 10 minutes, MCS with millions of iterations becomes computationally prohibitive for obtaining accurate failure probabilities. Therefore, the DGP+EFF method was adopted as a high-precision benchmark under stringent convergence criteria. For the 8-dimensional engineering case, conventional Kriging requires over 1,000 samples, while our previously recommended approach in [36] demands 500 samples; in contrast, the DGP model achieves accurate limit-state surface modeling with only 100 samples and controls the error within 1 %, demonstrating significant advantages in both efficiency and accuracy.

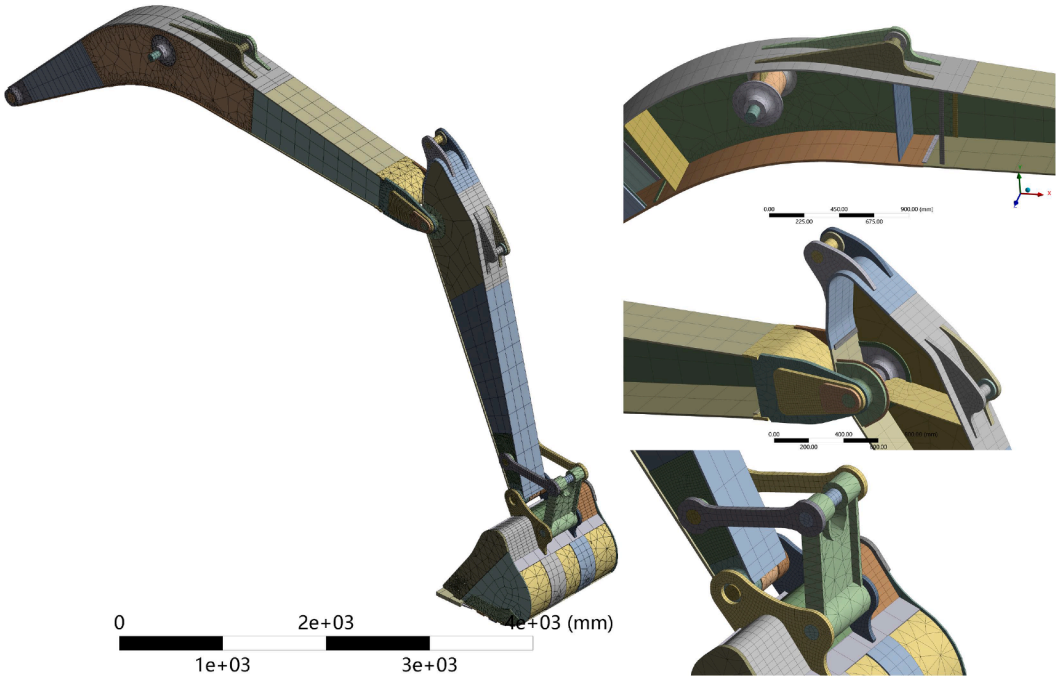
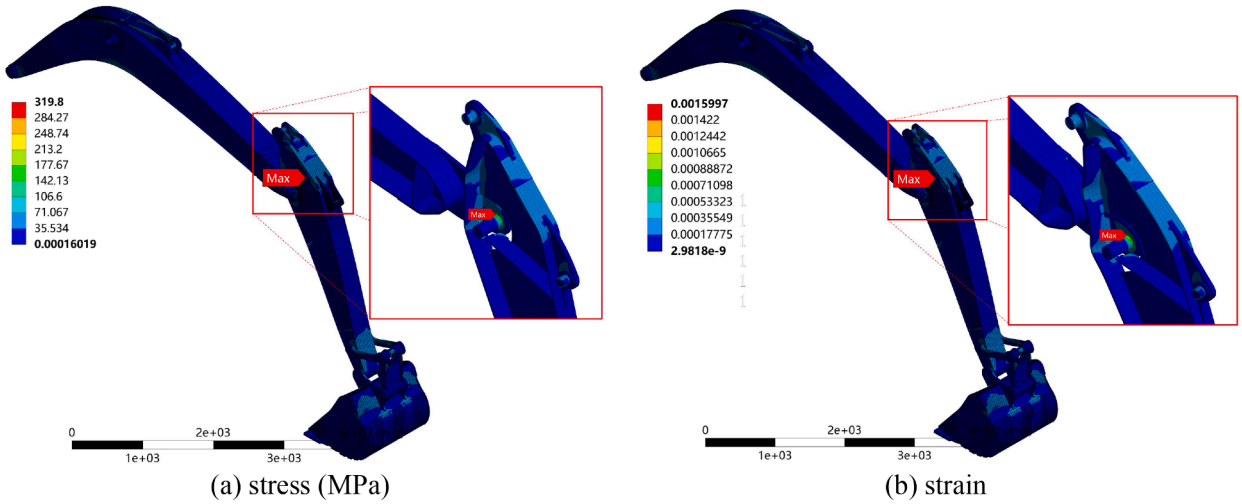


Fig. 10. Finite element mesh scheme.



(a) stress (MPa) (b) strain

Fig. 11. Contour plot from finite element analysis of hydraulic excavator structure.

Table 5
Parameters of the hydraulic excavator.

Parameter (units)	Mean	Standard deviation
F_{boom} (kN)	400	40
F_{arm} (kN)	200	20
F_{bucket} (kN)	160	16
d_1 (mm)	16	0.2
d_2 (mm)	16	0.2
d_3 (mm)	16	0.2
d_4 (mm)	12	0.2
d_5 (mm)	14	0.2

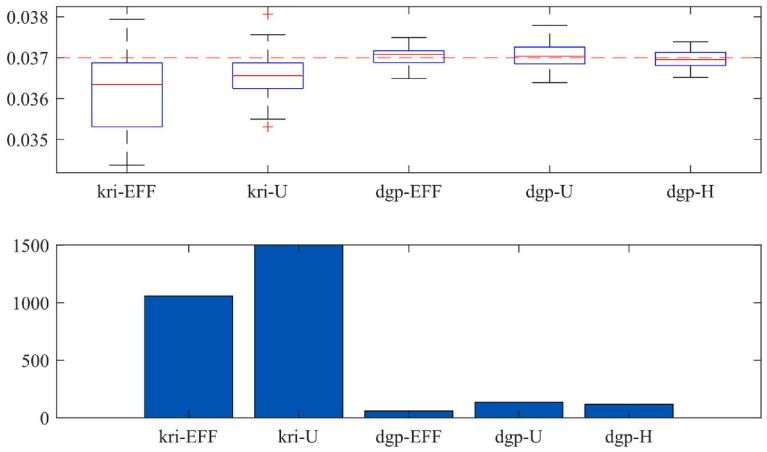


Fig. 12. Boxplots of all compared methods (Hydraulic excavator case).

Table 6
Results of the Hydraulic excavator.

Method	N_{call}	P_f	ε %	COV
AL-DGP1-MCS+EFF (Benchmark)	211.1	0.03700		0.010
AK-MCS+EFF	1059.2	0.03618	2.216	0.040
AK-MCS+U	1502.4	0.03653	1.270	0.042
AL-DGP1-MCS+EFF	60.3	0.03705	0.135	0.039
AL-DGP1-MCS+U	137.2	0.03704	0.108	0.040
AL-DGP1-MCS+H	118.1	0.03695	0.135	0.040
Liu et al., Eq. (7) in [36]	500.0	0.03810	2.973	-

6. Conclusions

This study addresses the challenges of high-dimensional input spaces and severely limited sample sizes in engineering machinery reliability analysis by proposing an active learning framework based on DGP and implementing it in a MATLAB toolbox. The hierarchical architecture of DGP enables automatic and flexible warping of the input space, fundamentally overcoming the stationarity limitation of conventional GP. This allows the model to effectively capture complex, non-stationary patterns often present in finite element simulation outputs. Furthermore, unlike computationally efficient but uncertainty-underestimating variational inference, the MCMC sampling scheme adopted in this work more comprehensively and accurately quantifies all sources of uncertainty, making it particularly suitable for small-sample scenarios in engineering applications. Numerical and engineering case studies demonstrate that the proposed DGP approach reduces the required number of training samples by 50-90 % (with particularly notable results in the excavator case study) while consistently maintaining estimation errors below 1 %. These results validate the practical utility of the proposed toolbox in structural reliability applications.

Although the advantages of the method may diminish in large-sample settings due to increased computational overhead and comparable accuracy with shallow surrogate models, its high sample efficiency aligns well with the small-data context typical of heavy machinery engineering, which motivated the development and promotion of this model. Future work will focus on developing customized active learning strategies that leverage the intrinsic properties of DGP and further investigating its effectiveness in reliability analysis for other types of engineering machinery.

CRediT authorship contribution statement

Yuan Lu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Yuxin Liu:** Writing – review & editing, Investigation, Data curation. **Hong-Zhong Huang:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Yan-Feng Li:** Project administration, Investigation, Formal analysis. **Zhe Deng:** Writing – review & editing, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The support of National Key R&D Program of China (No. 2023YFB3408501) is gratefully acknowledged.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.apm.2025.116477.

Appendix A. Toolbox details

Model construction

The configuration options for the fit_*_layerfunction series can be specified using name-value pairs, with available parameters as follows:

The 'normalize' (if set true) scales x to the unit hypercube and standardizes y to zero mean and unit variance. The 'nmcmc' sets the number of MCMC iterations (default: 10,000). A separable kernel can be activated via 'sep' (default: false). Progress during model fitting is displayed when 'veb' is true (default). Initial values for the nugget and length-scale hyperparameters are set by 'g_0′ (default: 0.001) and 'theta_0′ (default: 0.1), respectively. If the true nugget value is known, it is assigned to 'true_0′ (otherwise: empty). Additional settings are passed via a structure in settings (default: empty). The 'cov' parameter selects the covariance function, either 'matern' (default) or 'exp2′. For the Matern kernel, the smoothness is controlled by 'v' (default: 2.5).

fit = fit_two_layer(x, y)

fit = fit_three_layer(x, y)

For multi-layer DGP models, additional configuration parameters emerge: the latent layer dimensionality 'D' (default: input dimension *d*), initial latent variables 'w_0′ and 'z_0′ (default: empty) and their corresponding length scale hyperparameters 'theta_w_0′ and 'theta_z_0′ (default: 0.1).

The returned fit structure encapsulates the fitted DGP model. It uses layer for DGP layer count, x and y for fitted datasets, nmcmc for MCMC iterations, xsc and ysc (exist when 'normalize' is set to true) for per-column min-max boundaries of x and mean-standard deviation statistics of y, settings for basic configurations, v for Matern smoothness, g, theta, w, z for sampling values of hyperparameters and hidden layers, tau2 for sampling values of output scale, ll for log-likelihood trace.

Model prediction

Prediction behavior is configured via lite for computational efficiency (default: true), yielding only marginal variances rather than full covariance matrices; latent layer distributions can optionally persist through store_latent (default: false), while mean_map (default: true) enables conditional mean substitution to approximate hierarchical uncertainty; extended capabilities include return_all for MCMC prediction traces and Bayesian optimization utilities via EI for Expected Improvement quantification alongside entropy_limit constraints.

fit = predict(fit, xnew, 'store_latent', true, 'lite', false);

The returned structure fit preserves the original model specification while appending prediction-specific fields: mean stores the posterior predictive mean, s2 contains pointwise variances, and sigma retains the full covariance matrix (computed only when lite = false).

When normalization is enabled ('normalize' = true), the fitting procedure stores scaling parameters xsc and ysc within the fit structure. During subsequent prediction via predict, the function automatically detects the presence of fit.normalize and applies affine transformations: test inputs are projected to the unit hypercube using xsc, while predicted outputs undergo inverse scaling where the posterior mean is shifted by ysc(1) and scaled by ysc(2), and standard deviations are multiplied by ysc(2). The pseudo-code is shown in Table B.1.

The krig function requires two mandatory inputs: the response vector y and the precomputed squared Euclidean distance dx. It offers extensive configurability through name-value pairs: theta specifies length scale (default: 1), g defines the noise level (default: 0), tau2 sets output scale (default: 1), s2 controls pointwise variance computation (default: true), sigma enables full covariance output (default: false), f_min calculates the minimum predicted value at observed locations (default: false), v selects Matern smoothness or exponential kernel (default: 2.5), while prior_mean and prior_mean_new accept prior means for training and test points (default: 0), and d_new / d_cross supply test-data distance matrices when available. The function returns a structure containing: the predictive mean, conditional variances (if s2 = true), the full covariance matrix (if sigma = true), and the minimum in sample prediction f_min (if requested). An example of fitting a two-dimensional function is shown in Table B.2.

Model initialization and validation

This section formalizes the pre-execution verification framework critical for robust Deep Gaussian Process estimation, implementing four domain-specific validation routines: `check_inputs` performs fundamental integrity checks on regression data dimensions and preprocessing states, `check_ordering` enforces valid index sequencing for MCMC subspace traversal, `check_initialization` guarantees structural completeness of parameter initialization objects while imputing unspecified latent states via kernel interpolation; and `check_settings` systematically audits hyperparameter configurations against model topology constraints, enforcing internal consistency across hierarchy depth, warping transformations, and noise assumptions before sampling initialization.

check_inputs(x, y, true_g)

check_ordering(ordering, n)

The `check_inputs` and `check_ordering` functions return Boolean indicators, a value of false triggers immediate termination of the calling `fit_*_layer` routine with diagnostic error messaging.

initial = check_initialization(initial, layer, x, D, vecchia, v)

The `check_initialization` function returns a structure `initial` containing four critical initialization components: latent layer state, length scale, noise level, and output scale.

settings = check_settings(settings, layer, monowarp, noisy)

The `check_settings` function returns a validated configuration structure comprising sliding window boundary parameters for uniform proposal and gamma distribution shape-rate hyperparameter pairs.

Similarly, the `check_*` series functions are automatically invoked internally by `fit_*_layer` routines during model initialization.

Kernel function

This section formalizes the covariance kernel implementations essential for Deep Gaussian Process regression. The `Exp2` function computes stationary covariance matrices under the exponentiated quadratic form, incorporating optional diagonal noise augmentation for numerical stability. Complementarily, the `Matern` implementation provides three parameterizations of the Matern family, accommodating distinct smoothness regimes, while automatically handling auto-covariance terms through embedded noise adjustments during kernel matrix construction. During computational execution, kernel functions are automatically called to generate covariance matrices.

K = Exp2(distmat, tau2, theta, g)

K = Matern(distmat, tau2, theta, g, v)

where `distmat` is squared distance matrix.

The following are the calculation formulas for two kernel functions:

$$\mathrm{Exp2}(\mathbf{x}) = \exp(-\mathbf{r}) \tag{32}$$

$$\mathrm{Matern}(\mathbf{x}) = \begin{cases} \tau^2 \exp\left(-\sqrt{\mathbf{r}}\right), \nu = 0.5 \\ \tau^2 \exp\left(-\sqrt{3\mathbf{r}}\right)\left(1+\sqrt{3\mathbf{r}}\right), \nu = 1.5 \\ \tau^2 \exp\left(-\sqrt{5\mathbf{r}}\right)\left(1+\sqrt{5\mathbf{r}}+\frac{5}{3}\mathbf{r}\right), \nu = 2.5 \end{cases} \tag{33}$$

where $\mathbf{r} = dist(\mathbf{x})/\theta$.

Data availability

Data will be made available on request.

References

[1] S.Z. Gao, Y.K. Zhao, et al., Application of response surface method based on new strategy in structural reliability analysis, Structures 57 (2023) 105202.
[2] J.F. Liu, C. Jiang, Surrogate modeling for high dimensional uncertainty propagation via deep kernel polynomial chaos expansion, Appl. Math. Model. 122 (2023) 167–186.

[3] Y. Shi, Z.Z. Lu, et al., An adaptive multiple-Kriging-surrogate method for time-dependent reliability analysis, Appl. Math. Model. 70 (2019) 545–571.
[4] H.M. Qian, Y.F. Li, et al., Time-variant system reliability analysis method for a small failure probability problem, Reliab. Eng. Syst. Saf. 205 (2021) 107261.
[5] H.Y. Zhan, N.C. Xiao, A new active learning surrogate model for time- and space-dependent system reliability analysis, Reliab. Eng. Syst. Saf. 253 (2025) 110536.
[6] Z. Wang, A. Shafieezadeh, REAK: reliability analysis through error rate-based adaptive Kriging, Reliab. Eng. Syst. Saf. 182 (2019) 33–45.
[7] S. Yu, X. Wu, et al., A two-level surrogate framework for demand-objective time-variant reliability-based design optimization, Reliab. Eng. Syst. Saf. 244 (2024) 109924.
[8] R.B. Gramacy, D.W. Apley, Local Gaussian process approximation for large computer experiments, J. Comput. Graph. Stat. 24 (2) (2015) 561–578.
[9] C. Lee, K.W. Wang, et al., Partitioned active learning for heterogeneous systems, J. Comput. Inf. Sci. Eng. 23 (4) (2023) 041009.
[10] I. Aydogdu, Y. Wang, Scalable Bayesian optimization based on exploitation-enhanced sparse Gaussian process, Struct. Multidiscip. Optim. 67 (2024) 215.
[11] A. Damianou, N.D. Lawrence, Deep Gaussian Processes, in: Proceedings of the Artificial Intelligence and Statistics, 2013.
[12] T. Zhou, Y.B. Peng, Gaussian process regression based on deep neural network for reliability analysis in high dimensions, Struct. Multidiscipl. Optim. 66 (2023) 131.
[13] M. Sher, K. Shah, et al., Using deep Neural Network in computational analysis of coupled systems of fractional integro-differential equations, J. Comput. Appl. Math. (2025) 116912.
[14] M. Alqudah, K. Shah, et al., Mathematical modeling of psychological disease by using artificial intelligence tools, Fractals (2025).
[15] K. Shah, T. Abdeljawad, et al., Analyzing a coupled dynamical system of materials recycling in chemostat systems with artificial deep neural network, Model. Earth Syst. Environ. 11 (5) (2025) 1–4.
[16] D. Rajaram, T.G. Puranik, et al., Empirical assessment of Deep Gaussian Process surrogate models for engineering problems, J. Aircr. 58 (2021) 182–196.
[17] H.B. Yu, Y.Z. Chen, et al., Implicit posterior variational inference for Deep Gaussian Processes, in: Proceedings of the 33rd International Conference on Neural Information Processing Systems, 2019.
[18] L. Ding, R. Tuo, et al., A sparse expansion for Deep Gaussian Processes, IISE Trans 56 (2024) 559–572.
[19] A. Hebbal, L. Brevault, et al., Multi-fidelity modeling with different input domain definitions using Deep Gaussian Processes, Struct. Multidiscipl. Optim. 63 (2021) 2267–2288.
[20] X.L. Liu, S. Cui, et al., Remaining useful life prediction with uncertainty quantification for rotating machinery: A method based on explainable variational Deep Gaussian Process, Neurocomputing 638 (2025) 130232.
[21] T. Nakamura, T. Koriyama, et al., Sequence-to-sequence learning for deep Gaussian process based speech synthesis using self-attention GP layer, Interspeech, 2021, pp. 121–125.
[22] M.I. Radaideh, T. Kozlowski, Surrogate modeling of advanced computer simulations using Deep Gaussian Processes, Reliab. Eng. Syst. Saf. 195 (2020) 106731.
[23] J.J. Fei, J. Zhao, et al., Active learning methods with Deep Gaussian Processes, in: Proceedings of International Conference on Neural Information Processing, 2018.
[24] J. Yang, D. Klabjan, Bayesian active learning for choice models with Deep Gaussian Processes, IEEE Trans. Intell. Transp. Syst. 22 (2020) 1080–1092.
[25] H. Salimbeni, M. Deisenroth, Doubly stochastic variational inference for Deep Gaussian Processes, in: Proceedings of the Neural Information Processing Systems, 2017.
[26] H. Guo, J.L. Cai, et al., Predicting eddy current losses in large generator rotor using data-driven short connection Deep Gaussian Process, Eng. Appl. Artif. Intell. 158 (A) (2024) 111212.
[27] M. Havasi, J.M. Hernández-Lobato, et al., Inference in Deep Gaussian Processes using stochastic gradient Hamiltonian Monte Carlo, in: Proceedings of the Neural Information Processing Systems, 2018.
[28] A. Sauer, R.B. Gramacy, et al., Active learning for Deep Gaussian Process surrogates, Technometrics 65 (2023) 4–18.
[29] M. Kamiński, P. Świta, Structural stability and reliability of the underground steel tanks with the stochastic finite element method, Archiv. Civ. Mech. Eng. 15 (2015) 593–602.
[30] S.Y. Huang, S.H Zhang, et al., A new active learning Kriging metamodel for structural system reliability analysis with multiple failure modes, Reliab. Eng. Syst. Saf. 228 (2022) 108761.
[31] R.B. Gramacy, H.K.H. Lee, Bayesian treed Gaussian process models with an application to computer modeling, J. Am. Stat. Assoc. 103 (2008) 1119–1130.
[32] I. Murray, R.P. Adams, et al., Elliptical slice sampling, in: Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, 2010.
[33] B. Echard, N. Gayton, et al., AK-MCS: An active learning reliability method combining Kriging and Monte Carlo Simulation, Struct. Saf. 33 (2) (2011) 145–154.
[34] N.C. Xiao, H.Y. Zhan, et al., A new reliability method for small failure probability problems by combining the adaptive importance sampling and surrogate models, Comput. Methods Appl. Mech. Eng. 372 (2020) 113336.
[35] W. Fauriat, N. Gayton, AK-SYS: An adaptation of the AK-MCS method for system reliability, Reliab. Eng. Syst. Saf. 123 (2014) 137–144.
[36] Y.X. Liu, H.Z. Huang, et al., Reliability evaluation and optimal design of hydraulic excavator working device using integrated optimal machine learning algorithm, J. Mech. Sci. Technol. 39 (7) (2025) 3725–3734.